Especialização em Ciência
de Dados e Analytics

Processamento de Dados em Larga Escala

{ Hadoop MapReduce}

Prof. Jairson Rodrigues
jairson.rodrigues@univasf.edu.br

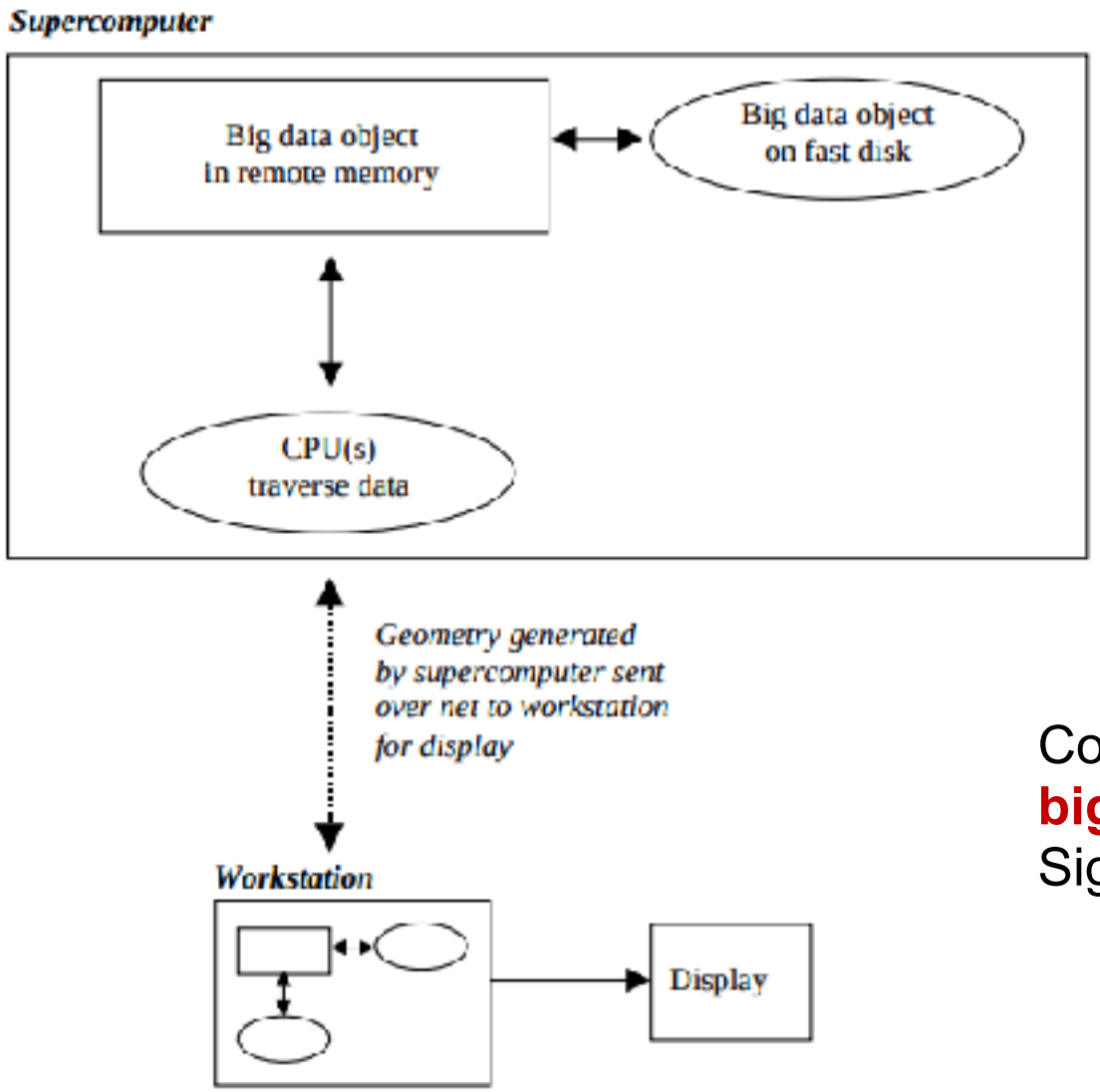
{ mapReduce }

grande
Hadoop
MapReduce
volume
velocidade
análise
Latu-Sensu
Especialização
HDFS
validade
viralidade
variedade
Poli-UPE
Ciência
dados
inteligência
máquina
aprendizagem
vocabulário
valor
viscosidade
velocidade

{ processamento de grandes volumes }

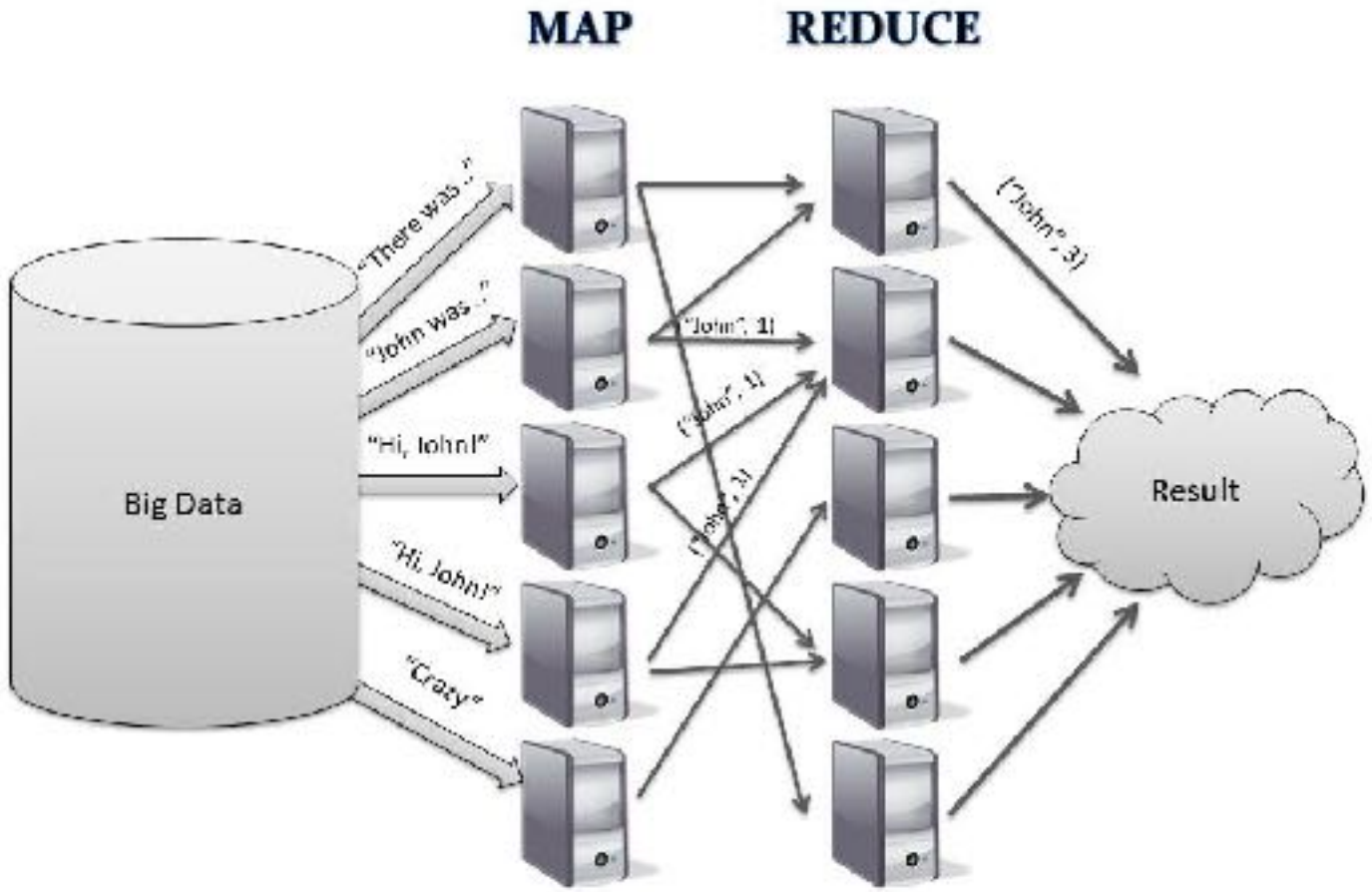
- Computação distribuída
  - Ambientes de memória compartilhada?
  - OpenMP?
  - Message Passage Interface (MPI)?
- Em todos os casos o programador deve se preocupar em gerenciar os recursos
  - mutexes, semáforos, detalhes da infraestrutura adjacente, dependência de plataforma, sistema operacional, linguagem...

{ mover a computação até o dado }

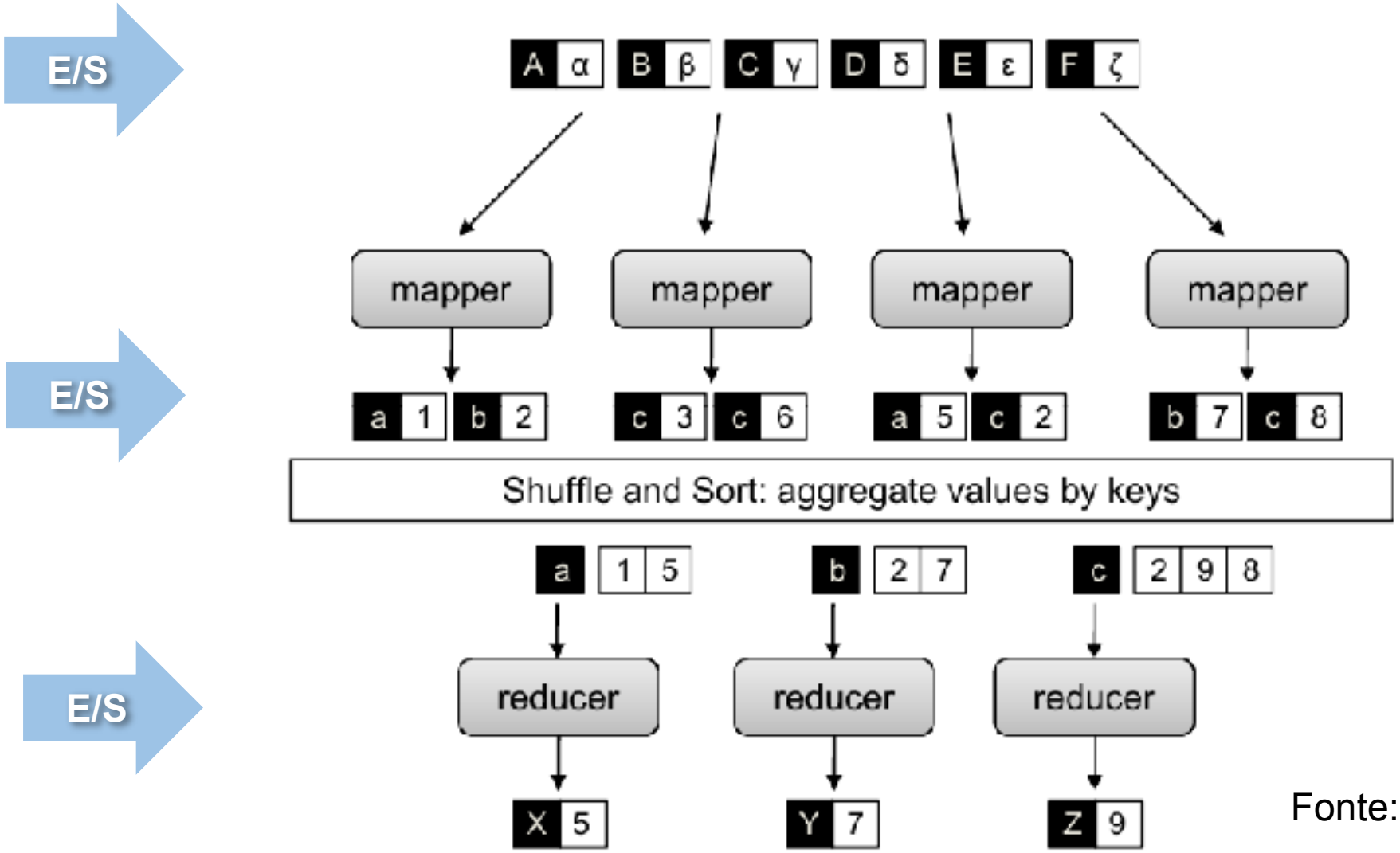


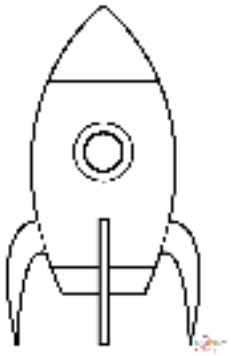
Cox, Michael, and David Ellsworth. "Managing **big data** for scientific visualization." ACM Siggraph. Vol. 97. **1997**.

{ mapreduce - visão simplificada }



{ mapreduce - outra visão simplificada }





SSD

Fonte: adaptado de [5]

{ mapreduce - modelo de programação }

- Opera exclusivamente em pares *<key,value>*
- Framework enxerga a entrada como conjunto de pares *<key,value>* e produz um outro conjunto de pares *<key, value>* como saída
- Inspirando fundamentalmente em paradigmas funcionais
- Formalmente:

$$\text{map: } (k1\ ,\ v1\ ) \rightarrow [(k2\ ,\ v2\ )]$$
$$\text{reduce: } (k2\ ,\ [v2\ ]) \rightarrow [(k3\ ,\ v3\ )]$$

{ princípios **bem básicos** de programação funcional [5] }

- Para compreender...
- Imagine somar todos os quadrados de uma lista de números
  - Entrada: 2, 3, 5, 7
  - Processamento: $2^2 + 3^2 + 5^2 + 7^2$
  - Saída: 87
- Vamos processar o mesmo problema com funções de alta ordem
  - map
  - fold (sim, fold)

{ princípios **bem básicos** de programação funcional [5] }

- Dada uma lista
  - map -> recebe como argumento uma função f (que recebe apenas um parâmetro) e a aplica a todos os elementos da lista
  - fold -> recebe como argumento uma função g e um valor inicial, que é aplicado à função g, cujo resultado é acumulado e novamente aplicado/acumulado ao segundo, terceiro e assim por diante a todos os elementos da lista.

{ exemplo ** }

- lista: 2, 3, 5, 7
  - map -> $f(x) = x^2$
  - fold -> g([f(x)], +)
- resultado do map

  [4, 9, 25, 49]
- aplicação do fold

  4

  4 + 9 = 13

  13 + 25 = 38

  38 + 49 = 87

'+' significa: função operação de adição

** notação adaptada para efeitos didáticos

{ uma abordagem map/reduce em python }

- x = [2, 3, 5, 7]
- y = map(lambda z: z**2, x)
- print y

  [4, 9, 25, 49]
- q = reduce(lambda m,n : m+n, y)
- print q

  87

{ mappers e reducers }

- Pares <key, value> formam a estrutura básica de um programa MapReduce
- Chaves e valores podem ser primitivas básicas como inteiros, floats, strings, bytes ou qualquer estrutura mais complexa, tais como listas, tuplas etc.
- Algumas analogias:
  - Para um conjunto de páginas web
    - key: URL
    - value: conteúdo HTML da página
  - Para um grafo
    - key: ID do nó
    - value: lista de nós adjacentes ao nó (key)

{ mappers e reducers }

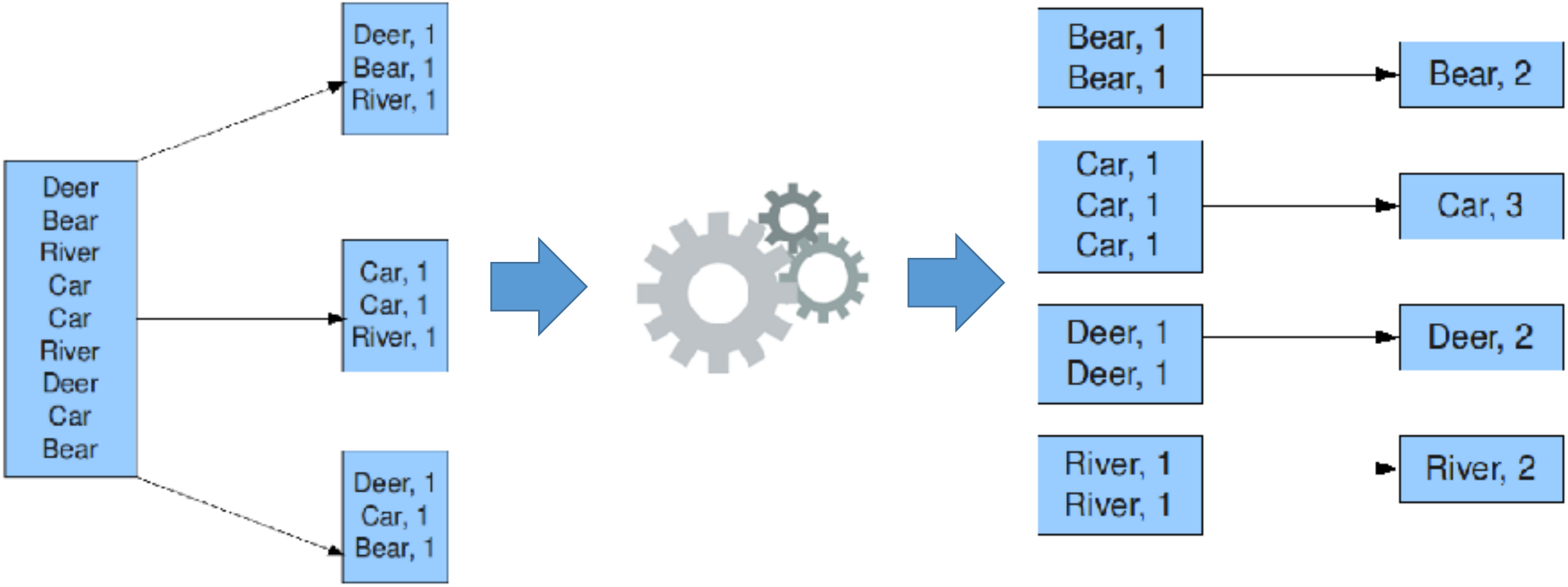
- map é aplicada a todo par <key, value> distribuído ao longo dos blocos do *filesystem*
- reduce é aplicada a todos os valores associados com chaves (keys) intermediárias geradas por map

{ "hello world" mapreduce / word count }

- Contar a ocorrência de cada palavra em um conjunto de textos
- Entrada: Projeto *Gutenberg* (demo didático com seis livros)
- Mapper: wc_map_**vx**.py
- Reducer: wc_reduce.py

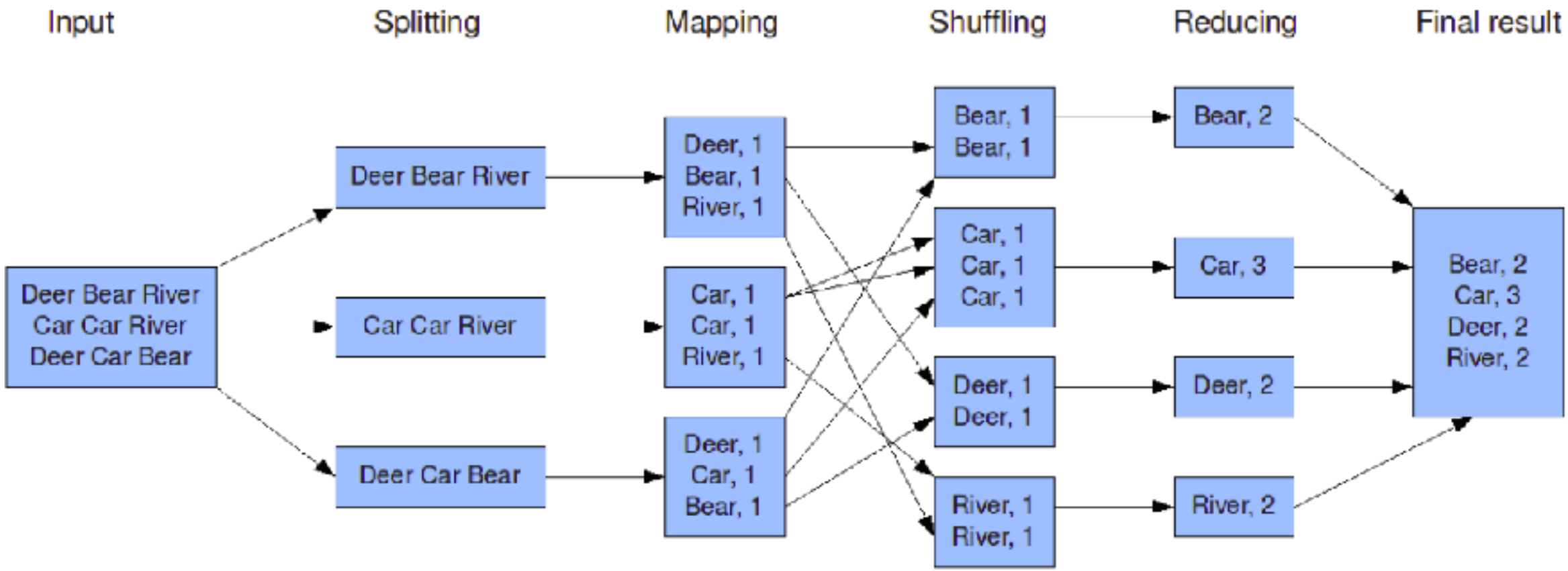
{ fase map }

{ fase reduce}



Fonte: https://blog.trifork.com/2009/08/04/introduction-to-hadoop/

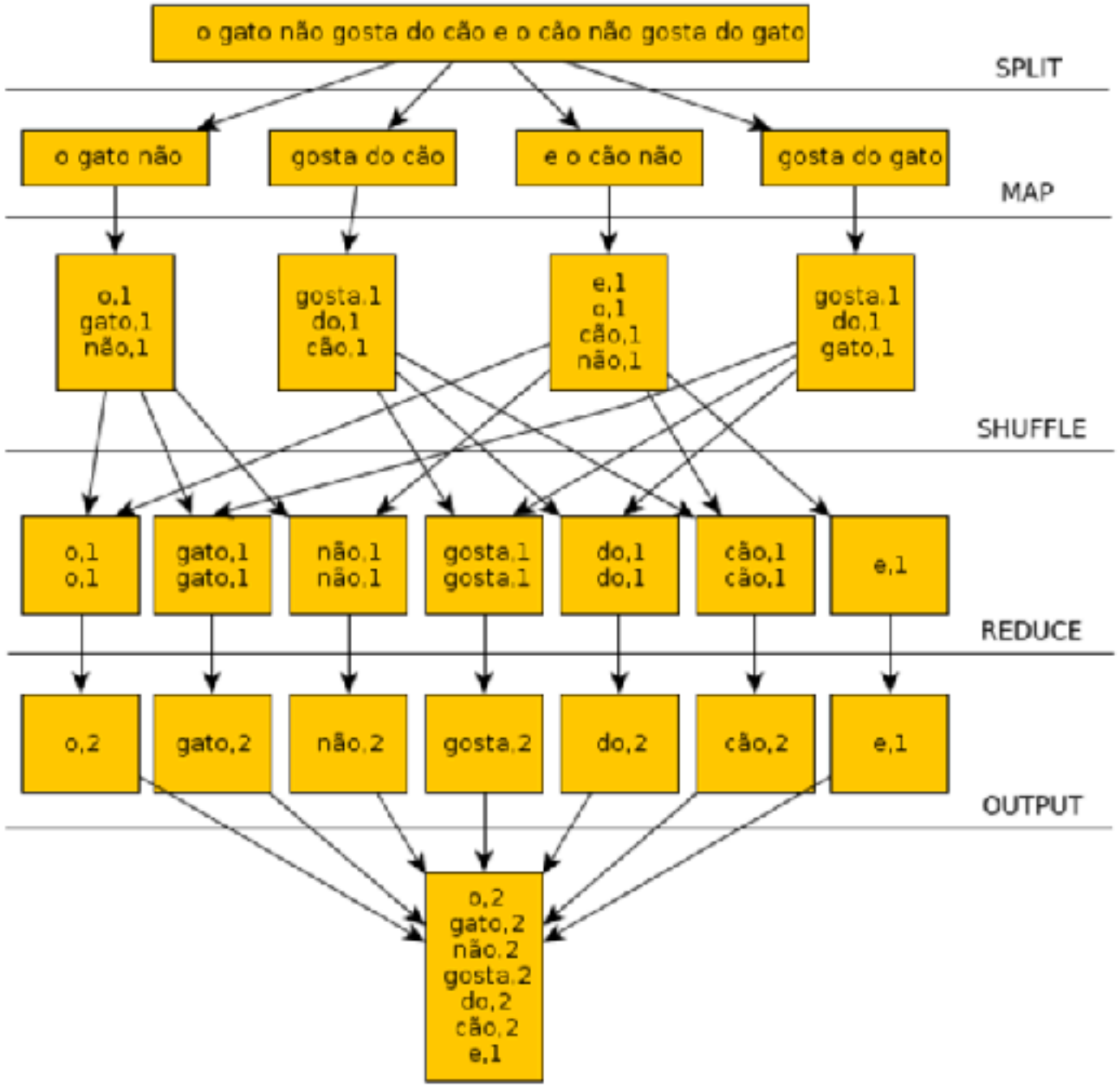
{ hello world mapreduce }



Fonte: https://blog.trifork.com/2009/08/04/introduction-to-hadoop/

{ hello world mapreduce (2) }

Fonte: Picoli, I. L., de Almeida, L. B., de Almeida, E. C. Otimização de Desempenho em Processamento de Consultas MapReduce.

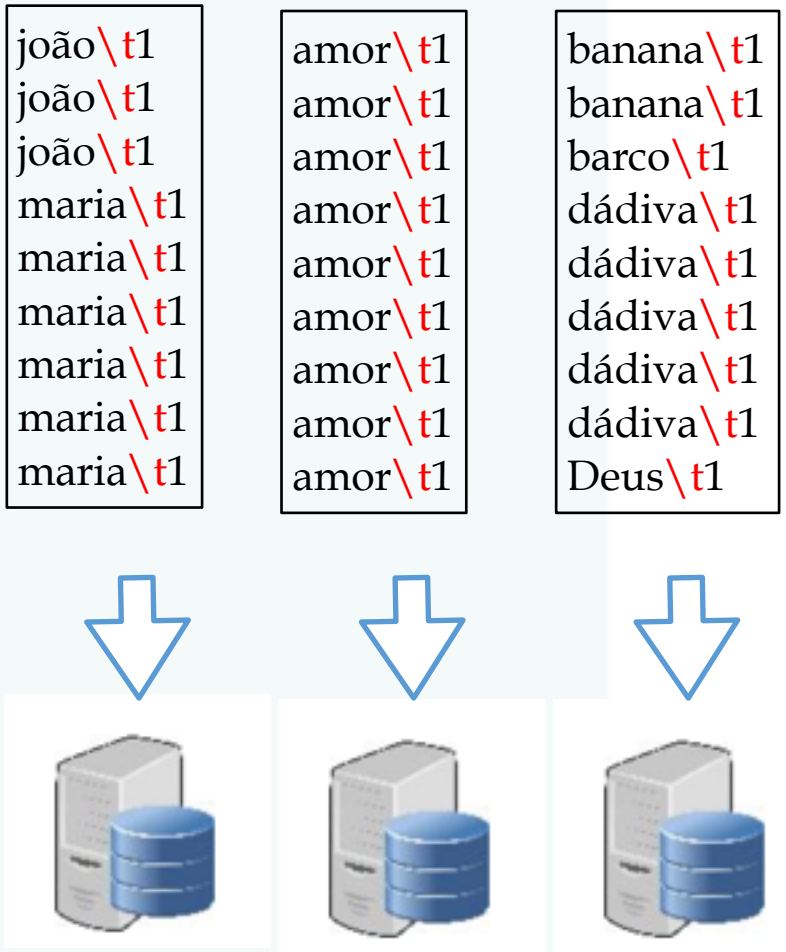


{ wc_map.py }

```
1   #!/usr/bin/env python
2   import sys
3
4   # toma linhas da entrada padrao
5   for line in sys.stdin:
6       # remove espacos em branco
7       line = line.strip()
8       # divide linha em palavras
9       words = line.split()
10
11      for word in words:
12          # escreve resultado na saida padrao
13          # que sera entrada para a fase reduce
14          # delimitador: tab
15          print '%s\t%s' % (word, 1)
```

{ wc_reduce.py }

```python
#!/usr/bin/env python

from operator import itemgetter
import sys

current_word = None
word = None
current_count = 0

# input comes from STDIN
for line in sys.stdin:

    # extrai <chave, valor> de cada linha
    word, count_str = line.strip().split('\t')
    count = int(count_str)

    # hadoop ordena saída map por chave
    if current_word == word:
        current_count += count
    else:
        if current_word:
            # escreve total por chave
            print '%s\t%s' % (current_word, current_count)
        current_count = count
        current_word = word

if current_word == word:
    # computa a última chave na entrada do datanode
    print '%s\t%s' % (current_word, current_count)
```

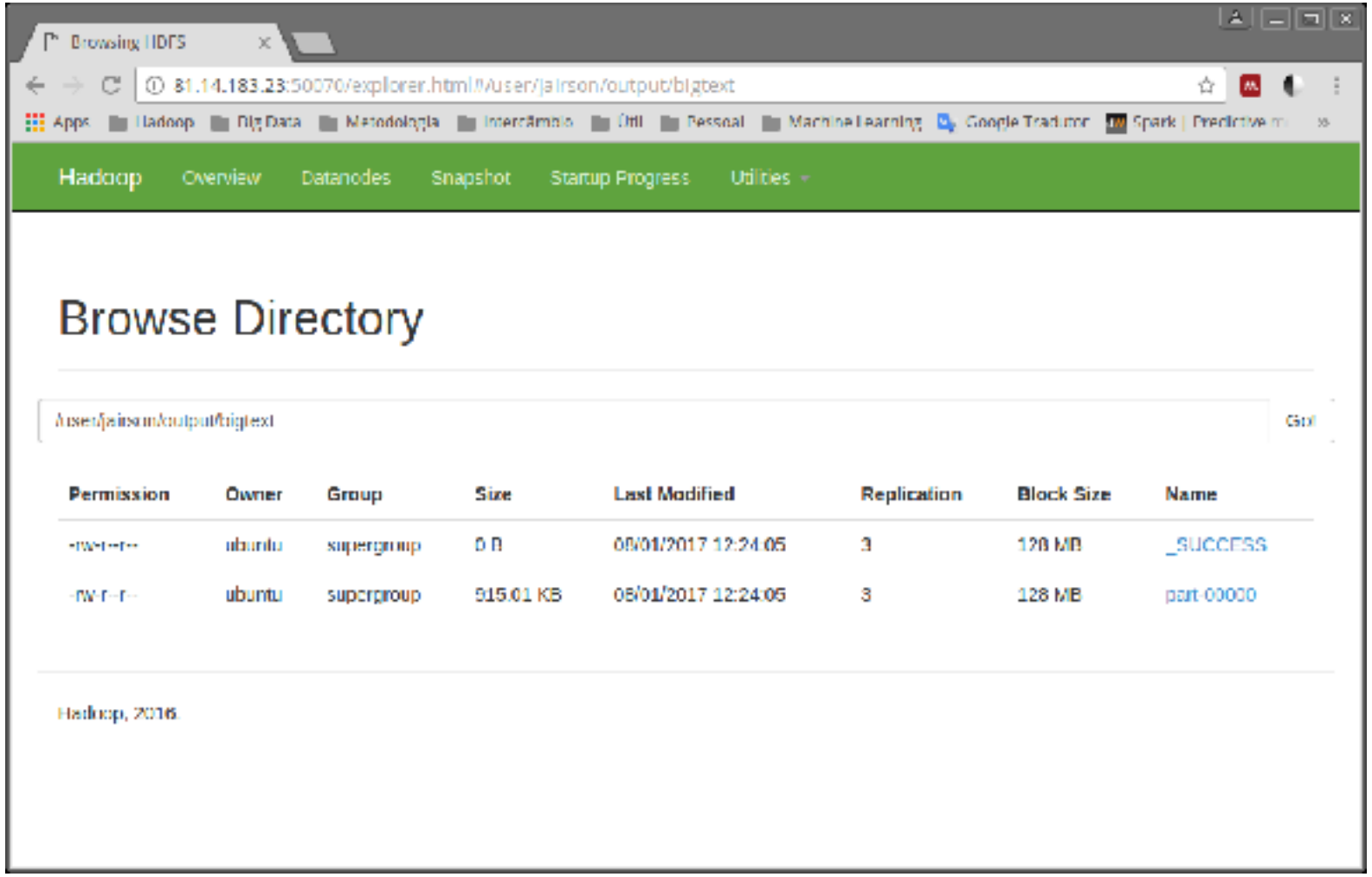


{ executando no cluster - entrada 1 }

```
cluster
ubuntu@name-node:~/python/wordcount$ hadoop jar /usr/local/hadoop/share/hadoop/tools/lib/hadoop-streaming-2.7.3.jar -file mapper.py -mapper
mapper.py -file reducer.py -reducer reducer.py -input /user/jairson/input/bigtext/* -output /user/jairson/output/bigtext/
17/01/08 15:23:44 WARN streaming.StreamJob: -file option is deprecated, please use generic option -files instead.
packageJobJar: [mapper.py, reducer.py, /tmp/hadoop-unjar2360298798677516218/] [] /tmp/streamjob1335253878811948279.jar tmpDir=null
17/01/08 15:23:45 INFO client.RMProxy: Connecting to ResourceManager at /192.168.4.221:8032
17/01/08 15:23:45 INFO client.RMProxy: Connecting to ResourceManager at /192.168.4.221:8032
17/01/08 15:23:46 INFO mapred.FileInputFormat: Total input paths to process : 1
17/01/08 15:23:46 INFO mapreduce.JobSubmitter: number of splits:2
17/01/08 15:23:46 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1482799310881_0017
17/01/08 15:23:47 INFO impl.YarnClientImpl: Submitted application application_1482799310881_0017
17/01/08 15:23:47 INFO mapreduce.Job: The url to track the job: http://name-node:8088/proxy/application_1482799310881_0017/
17/01/08 15:23:47 INFO mapreduce.Job: Running job: job_1482799310881_0017
17/01/08 15:23:52 INFO mapreduce.Job: Job job_1482799310881_0017 running in uber mode : false
17/01/08 15:23:52 INFO mapreduce.Job:  map 0% reduce 0%
17/01/08 15:23:58 INFO mapreduce.Job:  map 50% reduce 0%
17/01/08 15:23:59 INFO mapreduce.Job:  map 100% reduce 0%
17/01/08 15:24:06 INFO mapreduce.Job:  map 100% reduce 100%
17/01/08 15:24:07 INFO mapreduce.Job: Job job_1482799310881_0017 completed successfully
17/01/08 15:24:07 INFO mapreduce.Job: Counters: 56
        File System Counters
                FILE: Number of bytes read=1888216
                FILE: Number of bytes written=21976717
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=8492906
                HDFS: Number of bytes written=83697[illegible]
                HDFS: Number of read operations=9
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Killed map tasks=1
                Launched map tasks=2
                Launched reduce tasks=1
                Rack-local map tasks=2
                Total time spent by all maps in occupied slots (ms)=8963
                Total time spent by all reduces in occupied slots (ms)=5121
                Total time spent by all map tasks (ms)=8963
                Total time spent by all reduce tasks (ms)=5121
                Total vcore-milliseconds taken by all map tasks=8963
                Total vcore-milliseconds taken by all reduce tasks=5121
                Total megabyte-milliseconds taken by all map tasks=9178112
                Total megabyte-milliseconds taken by all reduce tasks=5243904
```

{ saída - entrada 1 }

```
"Why,    42
"Why,"   4
"Why... 1
"Why?    7
"Why?"   5
"Will    22
"Wine?   1
"Wish    1
"With    14
"Without          2
"Without,         1
"Witness          1
"Witness:         8
"Women   1
"Women's          1
"Women, 2
"Women,"          1
"Won't   3
"Wonderful!"      1
"Wonderful!...    1
"Worse   1
"Wostov!          1
"Wostov,          3
"Would   10
--More--
```



{ vamos consolidar a teoria }

Prática: MapReduce

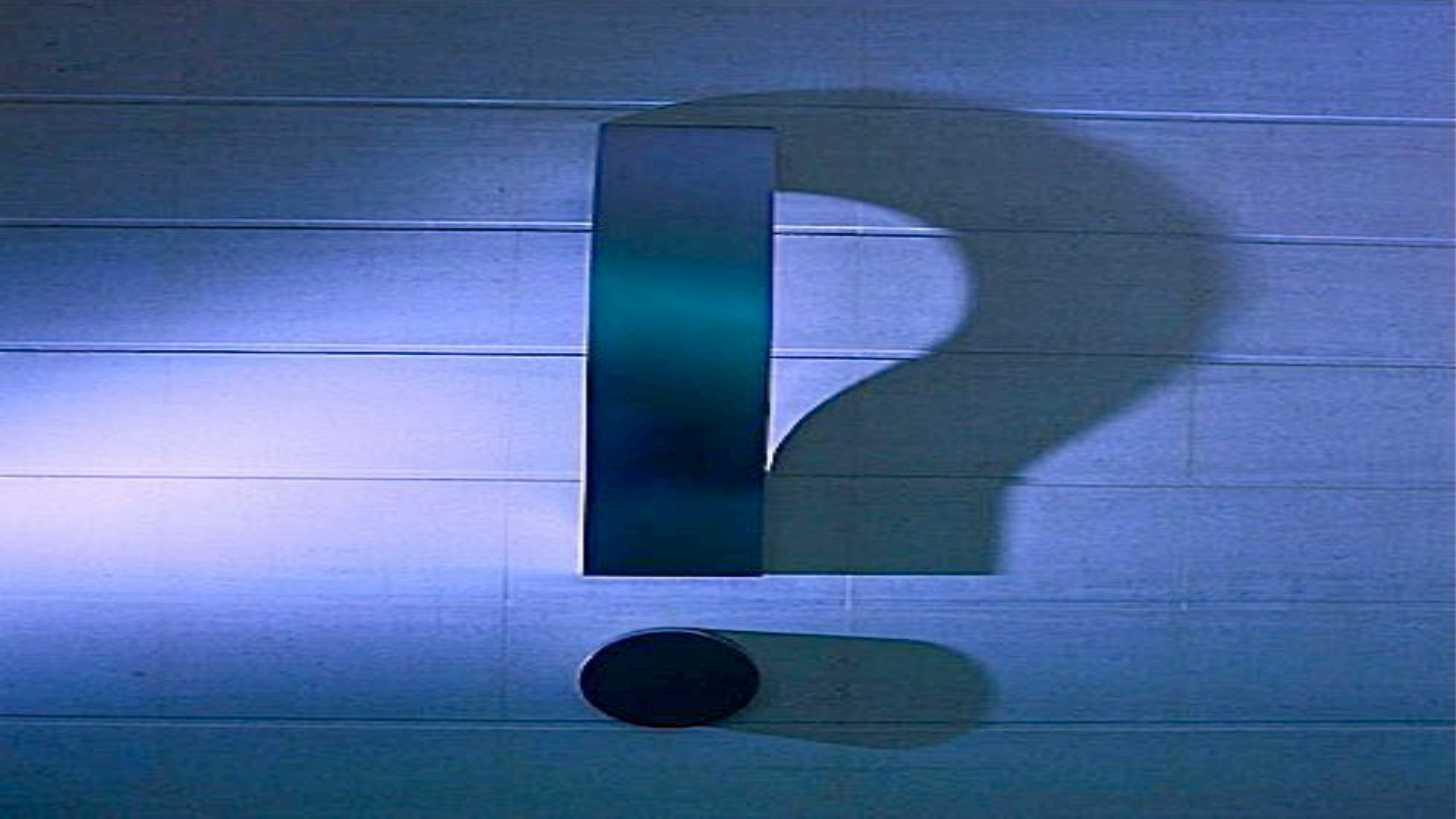
{ referências }

[1] Apache Hadoop Project - http://hadoop.apache.org/

[2] Who uses Hadoop? - https://wiki.apache.org/hadoop/PoweredBy

[3] Ghemawat, S., Gobioff, H., & Leung, S. T. (2003, October). The Google file system. In ACM SIGOPS operating systems review (Vol. 37, No. 5, pp. 29-43). ACM.

[4] J. Dean and S. Ghemawat, “MapReduce: simplified data processing on large clusters,” Commun. ACM, vol. 51, no. 1, p. 107, 2008.

[5] Lin, J., & Dyer, C. (2010). Data-intensive text processing with MapReduce. Synthesis Lectures on Human Language Technologies, 3(1), 1-177.

[6] P. Bedi, V. Jindal, and A. Gautam, “Beginning with big data simplified,” in 2014 International Conference on Data Mining and Intelligent Computing (ICDMIC), 2014, pp. 1–7.